Bikkad IT

Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Session 5.1: Python Tuples

Practice Questions with Python Code Answers

Compiled by Bikkad IT Institute | Teaching Notes

WHAT IS INSIDE

1. Create a Tuple
2. Accessing Items from a Tuple
3. Editing Items (Immutability)
4. Adding Items
5. Deletion
6. Operations on a Tuple
7. Tuple Functions
8. Difference Between List and Tuple
9. Tuple Unpacking
10. zip() Function with Tuples

60 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced

BEGINNER INTERMEDIATE ADVANCED

OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad

+91 77220 82226 Chh. Sambhajinagar

+91 77220 82227 Nashik

+91 77220 82224 Hadapsar, Pune

Session 5.1: Python Tuples - Practice Solutions

Prepared for: Bikkad IT Institute Students
Topics: Create a Tuple, Accessing Items from a Tuple, Editing Items (Immutability), Adding Items, Deletion, Operations on a Tuple, Tuple Functions, Difference Between List and Tuple, Tuple Unpacking, zip() Function with Tuples
Levels: 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
Date: October 2026

1. Create a Tuple

Q1. Store GPS Coordinates

BEGINNER

A delivery app stores the location of its Pune warehouse as latitude 18.5204 and longitude 73.8567. Store them in a tuple and print the tuple and its type.

Solution:

```
location = (18.5204, 73.8567)

print(location)
print(type(location))
```

Output:

```
(18.5204, 73.8567)
<class 'tuple'>
```

Q2. Days of the Week

BEGINNER

Create a tuple of the 7 days of the week (they never change). Print the tuple and the number of days using len().

Solution:

```
days = ("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")

print(days)
print(len(days))
```

Output:

```
('Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun')
7
```

Q3. Single-Item Tuple Trap

INTERMEDIATE

A survey has only one option "Yes". Create it as a tuple. Show that ("Yes") is a string but ("Yes",) with a comma is a tuple by printing both types.

Solution:

```
option1 = ("Yes")
option2 = ("Yes",)

print(type(option1))
print(type(option2))
```

Output:

```
<class 'str'>
<class 'tuple'>
```

Q4. Tuple from Other Data

INTERMEDIATE

Create tuples using tuple(): from the list ["Red", "Green", "Blue"], from the string "PUNE" and from range(1, 6).

Solution:

```
colours = tuple(["Red", "Green", "Blue"])
print(colours)

letters = tuple("PUNE")
print(letters)

numbers = tuple(range(1, 6))
print(numbers)
```

Output:

```
('Red', 'Green', 'Blue')
('P', 'U', 'N', 'E')
(1, 2, 3, 4, 5)
```

Q5. Tuple Without Brackets (Packing)

ADVANCED

Store an employee record "E101", "Sneha", 42000 WITHOUT writing brackets (tuple packing). Print it, its type, and create an empty tuple too.

Solution:

```
employee = "E101", "Sneha", 42000
print(employee)
print(type(employee))

empty = ()
print(empty)
print(len(empty))
```

Output:

```
('E101', 'Sneha', 42000)
<class 'tuple'>
()
0
```

Q6. Nested Tuple of Exam Timetable

ADVANCED

Create a nested tuple for an exam timetable where each inner tuple is (subject, date, time): ("Maths", "10-Nov", "10 AM"), ("Science", "12-Nov", "2 PM"), ("English", "14-Nov", "10 AM"). Print the full timetable and the number of exams.

Solution:

```
timetable = (
    ("Maths", "10-Nov", "10 AM"),
    ("Science", "12-Nov", "2 PM"),
    ("English", "14-Nov", "10 AM")
)

print(timetable)
print(len(timetable))
```

Output:

```
(('Maths', '10-Nov', '10 AM'), ('Science', '12-Nov', '2 PM'),
('English', '14-Nov', '10 AM'))
3
```

2. Accessing Items from a Tuple

Q7. First and Last Month

BEGINNER

A tuple stores the months of a financial year: ("Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec", "Jan", "Feb", "Mar"). Print the first and the last month.

Solution:

```
fy_months = ("Apr", "May", "Jun", "Jul", "Aug", "Sep",
             "Oct", "Nov", "Dec", "Jan", "Feb", "Mar")

print(fy_months[0])
print(fy_months[-1])
```

Output:

```
Apr
Mar
```

Q8. Quarter Using Slicing

BEGINNER

Using the same financial-year tuple, print the first quarter (first 3 months) and the last quarter (last 3 months) using slicing.

Solution:

```
fy_months = ("Apr", "May", "Jun", "Jul", "Aug", "Sep",
             "Oct", "Nov", "Dec", "Jan", "Feb", "Mar")

print(fy_months[:3])
print(fy_months[-3:])
```

Output:

```
('Apr', 'May', 'Jun')
('Jan', 'Feb', 'Mar')
```

Q9. Read an Exam Timetable

INTERMEDIATE

From the nested tuple timetable (("Maths", "10-Nov"), ("Science", "12-Nov"), ("English", "14-Nov")) print the date of the Science exam and the subject of the last exam.

Solution:

```
timetable = (("Maths", "10-Nov"),
             ("Science", "12-Nov"),
             ("English", "14-Nov"))

print(timetable[1][1])
print(timetable[-1][0])
```

Output:

```
12-Nov
English
```

Q10. Loop Through RGB Colour

INTERMEDIATE

A colour is stored as (255, 165, 0). Loop through the tuple and print each value with its index using enumerate().

Solution:

```
orange = (255, 165, 0)

for index, value in enumerate(orange):
    print(index, value)
```

Output:

```
0 255
1 165
2 0
```

Q11. Every Second Station

ADVANCED

A metro line has stations ("PCMC", "Sant Tukaram", "Bhosari", "Kasarwadi", "Phugewadi", "Dapodi", "Bopodi"). Print every second station, the route in reverse, and check whether "Dapodi" is on the line.

Solution:

```
stations = ("PCMC", "Sant Tukaram", "Bhosari", "Kasarwadi",
            "Phugewadi", "Dapodi", "Bopodi")

print(stations[::2])
print(stations[::-1])
print("Dapodi" in stations)
```

Output:

```
('PCMC', 'Bhosari', 'Phugewadi', 'Bopodi')
('Bopodi', 'Dapodi', 'Phugewadi', 'Kasarwadi', 'Bhosari', 'Sant
Tukaram', 'PCMC')
True
```

Q12. Find Position Safely

ADVANCED

In the tuple of blood groups ("A+", "B+", "O+", "AB+", "O-"), find the position of "O-" using index(). Then search for "B-" safely using in before calling index().

Solution:

```
groups = ("A+", "B+", "O+", "AB+", "O-")

print(groups.index("O-"))

search = "B-"
if search in groups:
    print(groups.index(search))
else:
    print(search, "not available")
```

Output:

```
4
B- not available
```

3. Editing Items (Immutability)

Q13. Try to Change a Tuple

BEGINNER

A tuple stores fixed GST slabs (5, 12, 18, 28). Try to change the first value to 6 inside try-except and print the error message to see that tuples are immutable.

Solution:

```
gst_slabs = (5, 12, 18, 28)

try:
    gst_slabs[0] = 6
except TypeError as e:
    print(e)

print(gst_slabs)
```

Output:

```
'tuple' object does not support item assignment
(5, 12, 18, 28)
```

Q14. Edit by Converting to List

BEGINNER

The tuple prices = (100, 200, 300) must change its middle value to 250. Convert to a list, edit, and convert back to a tuple.

Solution:

```
prices = (100, 200, 300)

temp = list(prices)
temp[1] = 250
prices = tuple(temp)

print(prices)
```

Output:

```
(100, 250, 300)
```

Q15. Rebuild a Tuple with Slicing

INTERMEDIATE

The team tuple ("Rohit", "Virat", "Rahul", "Hardik") must replace "Rahul" with "Shubman". Do it WITHOUT converting to a list, by joining slices.

Solution:

```
team = ("Rohit", "Virat", "Rahul", "Hardik")

team = team[:2] + ("Shubman",) + team[3:]
print(team)
```

Output:

```
('Rohit', 'Virat', 'Shubman', 'Hardik')
```

Q16. New Object After Edit

INTERMEDIATE

Show that "editing" a tuple actually creates a new tuple. Store id() before and after rebuilding (1, 2, 3) into (1, 20, 3) and compare.

Solution:

```
nums = (1, 2, 3)
old_id = id(nums)

nums = nums[:1] + (20,) + nums[2:]
new_id = id(nums)

print(nums)
print(old_id == new_id)
```

Output:

```
(1, 20, 3)
False
```

Q17. Mutable Item Inside a Tuple

ADVANCED

A tuple stores a student name and a list of marks: ("Kiran", [70, 80]). Show that you CAN add a mark to the inner list even though the tuple itself is immutable.

Solution:

```
student = ("Kiran", [70, 80])

student[1].append(90)
print(student)

try:
    student[0] = "Kunal"
except TypeError as e:
    print(e)
```

Output:

```
('Kiran', [70, 80, 90])
'tuple' object does not support item assignment
```

Q18. Update a Config Tuple

ADVANCED

A server config is stored as ("localhost", 8080, "DEBUG"). Create a new config where the port becomes 9090 and mode becomes "PROD" using unpacking, keeping the host same.

Solution:

```
config = ("localhost", 8080, "DEBUG")

host, port, mode = config
new_config = (host, 9090, "PROD")

print(config)
print(new_config)
```

Output:

```
('localhost', 8080, 'DEBUG')
('localhost', 9090, 'PROD')
```

4. Adding Items

Q19. Add a New City

BEGINNER

A tuple of metro cities is ("Mumbai", "Delhi", "Chennai"). Add "Kolkata" by joining another tuple with +.

Solution:

```
metros = ("Mumbai", "Delhi", "Chennai")

metros = metros + ("Kolkata",)
print(metros)
```

Output:

```
('Mumbai', 'Delhi', 'Chennai', 'Kolkata')
```

Q20. Add Many Items

BEGINNER

Add ("Pune", "Hyderabad") to the tuple ("Mumbai", "Delhi") and print the new tuple and its length.

Solution:

```
cities = ("Mumbai", "Delhi")

cities = cities + ("Pune", "Hyderabad")
print(cities)
print(len(cities))
```

Output:

```
('Mumbai', 'Delhi', 'Pune', 'Hyderabad')
4
```

Q21. Insert at the Beginning

INTERMEDIATE

A queue tuple is ("T2", "T3"). A priority token "T1" must be added at the start. Use tuple concatenation.

Solution:

```
queue = ("T2", "T3")

queue = ("T1",) + queue
print(queue)
```

Output:

```
('T1', 'T2', 'T3')
```

Q22. Insert in the Middle via List

INTERMEDIATE

Insert "Lunch" at index 2 in the schedule tuple ("Class1", "Class2", "Class3", "Class4") by converting to a list and using insert().

Solution:

```
schedule = ("Class1", "Class2", "Class3", "Class4")

temp = list(schedule)
temp.insert(2, "Lunch")
schedule = tuple(temp)

print(schedule)
```

Output:

```
('Class1', 'Class2', 'Lunch', 'Class3', 'Class4')
```

Q23. Build a Tuple in a Loop

ADVANCED

Build a tuple of the squares of 1 to 5 by adding one item at a time inside a loop (start with an empty tuple).

Solution:

```
squares = ()

for n in range(1, 6):
    squares = squares + (n * n,)

print(squares)
```

Output:

```
(1, 4, 9, 16, 25)
```

Q24. Add Records from a List

ADVANCED

Approved loan IDs are stored in the tuple ("L1", "L2"). New approvals ["L3", "L4", "L5"] come as a list. Add them to the tuple and print the total approvals.

Solution:

```
approved = ("L1", "L2")
new_approvals = ["L3", "L4", "L5"]

approved = approved + tuple(new_approvals)
print(approved)
print(len(approved))
```

Output:

```
('L1', 'L2', 'L3', 'L4', 'L5')
5
```

5. Deletion

Q25. Delete Whole Tuple

BEGINNER

Create a tuple of temporary OTPs ("4821", "9934"). Delete it using del and check that the name no longer exists using "otps" in dir().

Solution:

```
otps = ("4821", "9934")
print(otps)

del otps
print("otps" in dir())
```

Output:

```
('4821', '9934')
False
```

Q26. Cannot Delete One Item

BEGINNER

Try to delete the second item of ("Pen", "Pencil", "Eraser") with del inside try-except and print the error.

Solution:

```
items = ("Pen", "Pencil", "Eraser")

try:
    del items[1]
except TypeError as e:
    print(e)
```

Output:

```
'tuple' object doesn't support item deletion
```

Q27. Remove an Item via List

INTERMEDIATE

Remove "Pencil" from the tuple ("Pen", "Pencil", "Eraser") by converting to a list and using remove().

Solution:

```
items = ("Pen", "Pencil", "Eraser")

temp = list(items)
temp.remove("Pencil")
items = tuple(temp)

print(items)
```

Output:

```
('Pen', 'Eraser')
```

Q28. Remove by Position with Slicing

INTERMEDIATE

Remove the item at index 2 from (10, 20, 30, 40, 50) using slicing only.

Solution:

```
nums = (10, 20, 30, 40, 50)

nums = nums[:2] + nums[3:]
print(nums)
```

Output:

```
(10, 20, 40, 50)
```

Q29. Remove All Cancelled Bookings

ADVANCED

Bookings are ("B1", "CANCEL", "B2", "CANCEL", "B3"). Create a new tuple without any "CANCEL" entries using a generator inside tuple().

Solution:

```
bookings = ("B1", "CANCEL", "B2", "CANCEL", "B3")

active = tuple(b for b in bookings if b != "CANCEL")
print(active)
```

Output:

```
('B1', 'B2', 'B3')
```

Q30. Remove Duplicates Keeping Order

ADVANCED

Visitor IDs are (5, 3, 5, 8, 3, 9). Remove duplicates while keeping the first-seen order and return the result as a tuple.

Solution:

```
visitors = (5, 3, 5, 8, 3, 9)
unique = ()

for v in visitors:
    if v not in unique:
        unique = unique + (v,)

print(unique)
```

Output:

```
(5, 3, 8, 9)
```

6. Operations on a Tuple

Q31. Join Two Batches

BEGINNER

Morning batch ("Asha", "Bilal") and evening batch ("Chetan", "Diya"). Combine them using +.

Solution:

```
morning = ("Asha", "Bilal")
evening = ("Chetan", "Diya")

all_students = morning + evening
print(all_students)
```

Output:

```
('Asha', 'Bilal', 'Chetan', 'Diya')
```

Q32. Repeat a Pattern

BEGINNER

A traffic signal cycle is ("Red", "Green", "Yellow"). Repeat it 2 times using *.

Solution:

```
cycle = ("Red", "Green", "Yellow")

print(cycle * 2)
```

Output:

```
('Red', 'Green', 'Yellow', 'Red', 'Green', 'Yellow')
```

Q33. Membership Check

INTERMEDIATE

Holidays are stored as ("26-Jan", "15-Aug", "02-Oct", "25-Dec"). Check whether "15-Aug" and "01-May" are holidays.

Solution:

```
holidays = ("26-Jan", "15-Aug", "02-Oct", "25-Dec")

print("15-Aug" in holidays)
print("01-May" in holidays)
```

Output:

```
True
False
```

Q34. Loop and Total

INTERMEDIATE

Daily steps for a week are (8200, 6500, 10400, 7300, 9100, 12000, 5600). Use a for loop to find the total steps and the days with more than 9000 steps.

Solution:

```
steps = (8200, 6500, 10400, 7300, 9100, 12000, 5600)
total = 0

for s in steps:
    total += s
    if s > 9000:
        print(s)

print(total)
```

Output:

```
10400
9100
12000
59100
```

Q35. Compare Version Numbers

ADVANCED

Software versions are stored as tuples: current = (3, 11, 4), latest = (3, 12, 0). Compare them with < and == to check whether an update is needed (tuples compare item by item).

Solution:

```
current = (3, 11, 4)
latest = (3, 12, 0)

print(current < latest)
print(current == latest)

if current < latest:
    print("Update available")
```

Output:

```
True
False
Update available
```

Q36. Nested Tuple Report

ADVANCED

Sales by branch are (("Pune", 45000), ("Nashik", 32000), ("Nagpur", 51000)). Loop through it, print each branch with its sales, and find the branch with the highest sales.

Solution:

```
sales = (("Pune", 45000), ("Nashik", 32000), ("Nagpur", 51000))
best_branch = ""
best_amount = 0

for branch, amount in sales:
    print(branch, amount)
    if amount > best_amount:
        best_amount = amount
        best_branch = branch

print(best_branch)
```

Output:

```
Pune 45000
Nashik 32000
Nagpur 51000
Nagpur
```

7. Tuple Functions

Q37. Marks Summary

BEGINNER

Marks are (78, 85, 62, 90, 71). Print len(), min(), max() and sum().

Solution:

```
marks = (78, 85, 62, 90, 71)

print(len(marks))
print(min(marks))
print(max(marks))
print(sum(marks))
```

Output:

```
5
62
90
386
```

Q38. Sorted Copy

BEGINNER

Prices are (499, 99, 1299, 299). Use sorted() to get a sorted list (ascending and descending). Show that the tuple itself is unchanged.

Solution:

```
prices = (499, 99, 1299, 299)

print(sorted(prices))
print(sorted(prices, reverse=True))
print(prices)
```

Output:

```
[99, 299, 499, 1299]
[1299, 499, 299, 99]
(499, 99, 1299, 299)
```

Q39. count() and index()

INTERMEDIATE

Ratings received are (5, 4, 5, 3, 5, 4). Count how many 5-star ratings there are and find the first position of the rating 3.

Solution:

```
ratings = (5, 4, 5, 3, 5, 4)

print(ratings.count(5))
print(ratings.index(3))
```

Output:

```
3
3
```

Q40. any() and all()

INTERMEDIATE

Daily attendance flags are (True, True, False, True). Use all() to check if the student attended every day and any() to check if he attended at least one day.

Solution:

```
attendance = (True, True, False, True)

print(all(attendance))
print(any(attendance))
```

Output:

```
False
True
```

Q41. Average and Above-Average

ADVANCED

Temperatures are (31.5, 33.0, 29.8, 34.2, 32.1). Find the average (rounded to 2 decimals) and make a tuple of temperatures above average.

Solution:

```
temps = (31.5, 33.0, 29.8, 34.2, 32.1)

average = round(sum(temps) / len(temps), 2)
print(average)

above = tuple(t for t in temps if t > average)
print(above)
```

Output:

```
32.12
(33.0, 34.2)
```

Q42. Sort Records by a Field

ADVANCED

Players are (("Ravi", 45), ("Amit", 72), ("Joy", 58)) as (name, score). Use sorted() with key to sort by score (highest first). Use a small function as the key.

Solution:

```
players = (("Ravi", 45), ("Amit", 72), ("Joy", 58))

def get_score(player):
    return player[1]

ranking = sorted(players, key=get_score, reverse=True)
print(ranking)
print(ranking[0][0])
```

Output:

```
[('Amit', 72), ('Joy', 58), ('Ravi', 45)]
Amit
```

8. Difference Between List and Tuple

Q43. Mutable vs Immutable

BEGINNER

Create a list and a tuple with the values 1, 2, 3. Change the first value of the list to 100. Try the same on the tuple inside try-except.

Solution:

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3)

my_list[0] = 100
print(my_list)

try:
    my_tuple[0] = 100
except TypeError as e:
    print(e)
```

Output:

```
[100, 2, 3]
'tuple' object does not support item assignment
```

Q44. Brackets and Type

BEGINNER

Create cart_items as a list and weekdays as a tuple. Print their types and explain in a comment why weekdays should be a tuple.

Solution:

```
cart_items = ["Shoes", "Bag"]
weekdays = ("Mon", "Tue", "Wed", "Thu", "Fri")

print(type(cart_items))
print(type(weekdays))
# Weekdays never change, so a tuple protects
# them from accidental edits
```

Output:

```
<class 'list'>
<class 'tuple'>
```

Q45. Memory Size Comparison

INTERMEDIATE

Use sys.getsizeof() to compare the memory used by a list and a tuple with the same 5 numbers.

Solution:

```
import sys

data_list = [10, 20, 30, 40, 50]
data_tuple = (10, 20, 30, 40, 50)

print(sys.getsizeof(data_list))
print(sys.getsizeof(data_tuple))
print(sys.getsizeof(data_tuple) < sys.getsizeof(data_list))
```

Output:

```
104
80
True
```

Q46. Available Methods

INTERMEDIATE

Count how many public methods a list has compared to a tuple using dir() and a comprehension that skips names starting with "_". Print the tuple methods.

Solution:

```
list_methods = [m for m in dir(list) if not m.startswith("_")]
tuple_methods = [m for m in dir(tuple) if not m.startswith("_")]

print(len(list_methods))
print(len(tuple_methods))
print(tuple_methods)
```

Output:

```
11
2
['count', 'index']
```

Q47. Tuple as Dictionary Key

ADVANCED

Store distances between city pairs in a dictionary using tuples as keys: ("Pune", "Mumbai") = 150, ("Pune", "Nashik") = 210. Then try a list as a key inside try-except.

Solution:

```
distance = {
    ("Pune", "Mumbai"): 150,
    ("Pune", "Nashik"): 210
}
print(distance[("Pune", "Nashik")])

try:
    bad = {["Pune", "Goa"]: 450}
except TypeError as e:
    print(e)
```

Output:

```
210
unhashable type: 'list'
```

Q48. Speed Test

ADVANCED

Use timeit to compare how fast Python creates a list vs a tuple of 5 numbers 1,00,000 times. Print which one is faster.

Solution:

```
import timeit

list_time = timeit.timeit("[1, 2, 3, 4, 5]", number=100000)
tuple_time = timeit.timeit("(1, 2, 3, 4, 5)", number=100000)

print(tuple_time < list_time)
```

Sample Output (changes on every run):

```
True
```

9. Tuple Unpacking

Q49. Unpack a Student Record

BEGINNER

Unpack the tuple ("Neha", 21, "Pune") into name, age and city variables and print them.

Solution:

```
student = ("Neha", 21, "Pune")

name, age, city = student
print(name)
print(age)
print(city)
```

Output:

```
Neha
21
Pune
```

Q50. Swap Two Values

BEGINNER

Swap the opening and closing balances a = 5000 and b = 7200 using tuple unpacking.

Solution:

```
a = 5000
b = 7200

a, b = b, a
print(a)
print(b)
```

Output:

```
7200
5000
```

Q51. Collect the Rest with *

INTERMEDIATE

A race result tuple is ("Gold-Anil", "Silver-Bina", "Ravi", "Sita", "Mohan"). Unpack the first two winners and collect the rest into a list using *.

Solution:

```
result = ("Gold-Anil", "Silver-Bina", "Ravi", "Sita", "Mohan")

first, second, *others = result
print(first)
print(second)
print(others)
```

Output:

```
Gold-Anil
Silver-Bina
['Ravi', 'Sita', 'Mohan']
```

Q52. Unpack Inside a Loop

INTERMEDIATE

Orders are (("Pizza", 2, 250), ("Burger", 3, 120)) as (item, quantity, price). Unpack each tuple in a for loop and print the line total.

Solution:

```
orders = (("Pizza", 2, 250), ("Burger", 3, 120))

for item, qty, price in orders:
    print(item, qty * price)
```

Output:

```
Pizza 500
Burger 360
```

Q53. Function Returning Multiple Values

ADVANCED

Write a function stats(numbers) that returns the minimum, maximum and average as a tuple. Unpack the result for the marks (56, 78, 91, 64).

Solution:

```
def stats(numbers):
    low = min(numbers)
    high = max(numbers)
    avg = sum(numbers) / len(numbers)
    return low, high, avg

lowest, highest, average = stats((56, 78, 91, 64))
print(lowest)
print(highest)
print(average)
```

Output:

```
56
91
72.25
```

Q54. Ignore Values with _ and Nested Unpacking

ADVANCED

A record is ("EMP7", ("Ajay", "Patil"), "IT", 61000). Unpack it to get the first name, last name and salary only. Use _ for values you don't need.

Solution:

```
record = ("EMP7", ("Ajay", "Patil"), "IT", 61000)

_, (first, last), _, salary = record
print(first)
print(last)
print(salary)
```

Output:

```
Ajay
Patil
61000
```

10. zip() Function with Tuples

Q55. Pair Products with Prices

BEGINNER

Products are ("Tea", "Coffee", "Milk") and prices are (15, 25, 30). Use zip() to create a tuple of pairs.

Solution:

```
products = ("Tea", "Coffee", "Milk")
prices = (15, 25, 30)

pairs = tuple(zip(products, prices))
print(pairs)
```

Output:

```
(('Tea', 15), ('Coffee', 25), ('Milk', 30))
```

Q56. Print Roll Number with Name

BEGINNER

Roll numbers (101, 102, 103) and names ("Om", "Pia", "Raj"). Use zip() in a for loop to print each roll number with the name.

Solution:

```
rolls = (101, 102, 103)
names = ("Om", "Pia", "Raj")

for r, n in zip(rolls, names):
    print(r, n)
```

Output:

```
101 Om
102 Pia
103 Raj
```

Q57. Total Bill per Item

INTERMEDIATE

Quantities (2, 1, 4) and rates (120, 450, 35) are stored in two tuples. Use zip() to calculate the amount for each item and the total bill.

Solution:

```
quantities = (2, 1, 4)
rates = (120, 450, 35)
total = 0

for q, r in zip(quantities, rates):
    amount = q * r
    print(amount)
    total += amount

print(total)
```

Output:

```
240
450
140
830
```

Q58. Unzip Pairs

INTERMEDIATE

A tuple of (city, temperature) pairs is (("Pune", 31), ("Goa", 33), ("Leh", 12)). Separate it into a cities tuple and a temperatures tuple using zip(*).

Solution:

```
data = (("Pune", 31), ("Goa", 33), ("Leh", 12))

cities, temps = zip(*data)
print(cities)
print(temps)
```

Output:

```
('Pune', 'Goa', 'Leh')
(31, 33, 12)
```

Q59. Daily Change in Stock Price

ADVANCED

Closing prices are (520, 535, 528, 541, 560). Use zip() of the tuple with itself shifted by one (prices, prices[1:]) to find the change from each day to the next.

Solution:

```
prices = (520, 535, 528, 541, 560)

changes = tuple(b - a for a, b in zip(prices, prices[1:]))
print(changes)
```

Output:

```
(15, -7, 13, 19)
```

Q60. Build a Dictionary and Find Topper

ADVANCED

Names ("Anu", "Bala", "Chris") and marks (88, 95, 79). Build a dictionary using dict(zip()) and find the topper using max() with key.

Solution:

```
names = ("Anu", "Bala", "Chris")
marks = (88, 95, 79)

result = dict(zip(names, marks))
print(result)

topper = max(result, key=result.get)
print(topper)
```

Output:

```
{'Anu': 88, 'Bala': 95, 'Chris': 79}
Bala
```

End of Solutions

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: October 2026